Designing and Analysis of Algorithms

Course Code: ECS 5101/CS514

- Dr Rahul Mishra
- IIT Patna

- **Lecture 11**

Recurrences

Recurrences in the context of computer science and mathematics refer to mathematical relationships or equations that describe the recurrence of a function or sequence in terms of its previous values. These equations express a recursive relationship, defining the value of a function for a certain input in terms of its values for smaller inputs. These are often used to analyze the time complexity of recursive algorithms.

Key Components of Recurrences:

1. **Base Case:** A recurrence relation typically includes a base case, which represents the starting point or condition where the recurrence stops. The solution for the base case is usually provided explicitly.
2. **Recursive Case:** The recurrence relation defines how to compute the value of the function for a given input based on the values of the function for smaller inputs. This is often expressed as a formula or equation involving the function's previous values.
3. **Initial Conditions:** Along with the base case, initial conditions may be specified to determine the values of the function for small input sizes where the recurrence relation alone might not apply.

Consider the Fibonacci sequence, where $F(n)$ represents the n^{th} Fibonacci number: $F(n) = F(n-1) + F(n-2)$ with base cases: $F(0) = 0, F(1) = 1$. In this example, the recurrence relation expresses the n^{th} Fibonacci number as the sum of the two preceding Fibonacci numbers. The base cases specify the starting values for $F(0)$ and $F(1)$.

Importance in Algorithm Analysis:

Recurrence relations are fundamental in the analysis of recursive algorithms. The time complexity of many recursive algorithms can be expressed through recurrence relations. Understanding and solving these recurrences help in predicting and analyzing the efficiency of algorithms, providing insights into their behavior as the input size grows. The recurrences play a crucial role in modeling and analyzing the recursive behavior of functions and algorithms, providing a powerful tool for expressing and understanding repetitive patterns in mathematical and computational contexts.

Recurrences

Example Factorial Function

Consider the factorial function $n!$, defined as the product of all positive integers up to n. The recurrence relation for factorial is as follows:

$$n! = n \times (n-1)!.$$

With the base case $0! = 1$. This recurrence relation expresses the factorial of n in terms of the factorial of $(n-1)$. The solution to this recurrence is $n! = n \times (n-1)!$, and it is used in many recursive algorithms.

Example Fibonacci Sequence

The Fibonacci sequence is defined as follows:

$$F(n) = F(n-1) + F(n-2).$$

With base cases $F(0) = 0$ and $F(1) = 1$. This recurrence relation expresses the n^{th} Fibonacci number in terms of the two preceding Fibonacci numbers. Solving this recurrence gives the well-known Fibonacci sequence: $0, 1, 1, 2, 3, 5, 8, 13, \cdots$

Example Merge Sort

Consider the recurrence relation for the time complexity of the Merge Sort algorithm:

$$T(n) = 2T\left(\frac{n}{2}\right) + n.$$

This recurrence expresses the time complexity $T(n)$ of sorting an array of size n in terms of sorting two halves of size $\frac{n}{2}$ and merging them. The solution to this recurrence is $T(n) = O(n \log n)$, making Merge Sort an efficient sorting algorithm.

Example Binary Search

The recurrence relation for the time complexity of Binary Search can be expressed as:

$$T(n) = T\left(\frac{n}{2}\right) + 1.$$

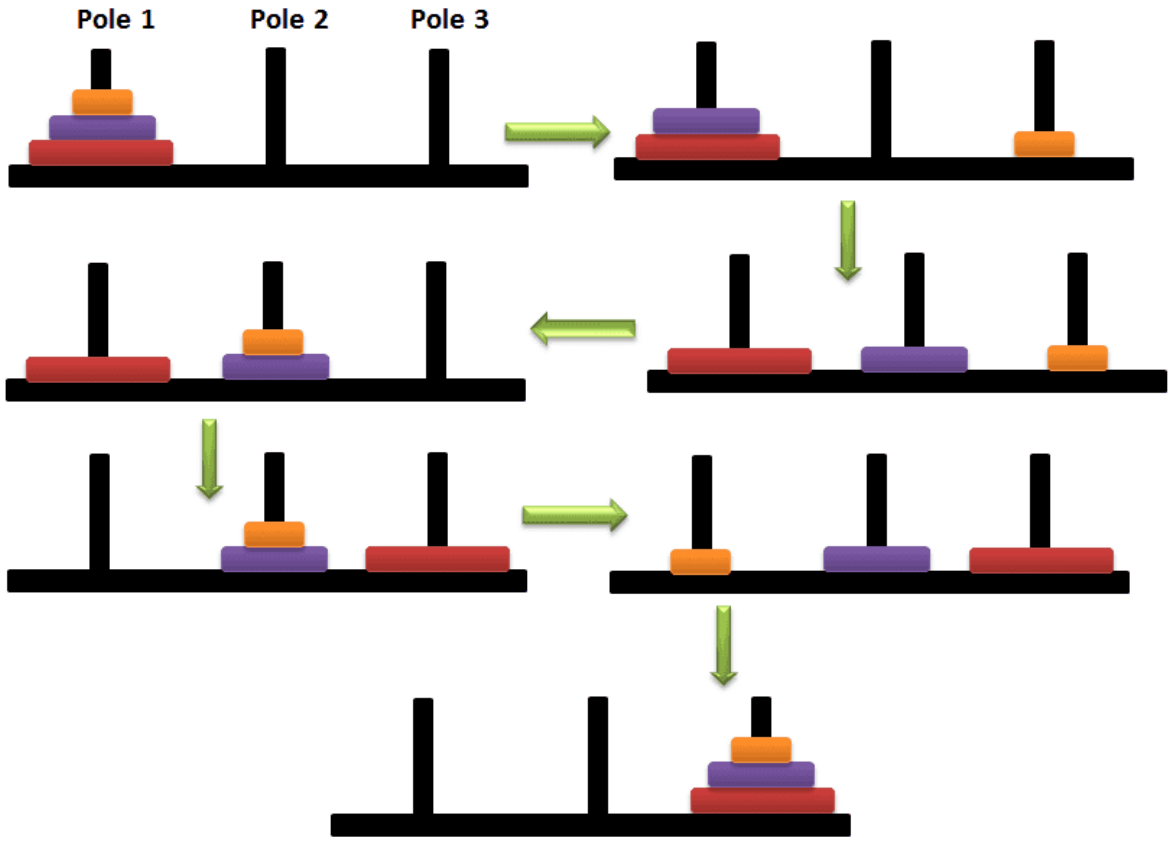
Recurrences

Example **Towers of Hanoi**

The recurrence relation for the number of moves required to solve the Towers of Hanoi problem with n disks is:

$$T(n) = 2T(n-1) + 1.$$

With the base case $T(1) = 1$. This recurrence expresses the number of moves needed to solve the problem for n disks in terms of solving it for $n-1$ disks twice and adding one additional move. The solution is $T(n) = 2^n - 1$



Detailed in Stack Chapter

Recurrences

There are several methods for solving recurrence relations, such as:

- **Substitution Method:** Guessing a solution and then proving it using mathematical induction.
- **Recurrence Tree Method:** Drawing a tree to represent the recursive calls and summing up the costs.
- **Master Theorem:** A specialized method for recurrence relations that fit a specific form.

Master Theorem

The master method is a formula for solving recurrence relations of the form:

T(n) = aT(n/b) + f(n),

where, **n** = size of input
a = number of sub-problems in the recursion
n/b = size of each sub-problem.
***All sub-problems are assumed to have the same size*. f(n) =** cost of the work done outside the recursive call, which includes the cost of dividing the problem and the cost of merging the solutions

Here**, a ≥ 1 and b > 1** are constants, and f(n) is an asymptotically positive function.
An asymptotically positive function means that for a sufficiently large value of **n**, we have **f(n) > 0.**

Master Theorem

Master Theorem

If $a \geq 1$ and $b > 1$ are constants and $f(n)$ is an asymptotically positive function, then the time complexity of a recursive relation is given by

$$T(n) = aT(n/b) + f(n)$$

where, T(n) has the following asymptotic bounds:

1. If $f(n) = O(n^{\log_b a-\epsilon})$, then $T(n) = \theta(n^{\log_b a})$.
2. If $f(n) = \theta(n^{\log_b a})$, then $T(n) = \theta(n^{\log_b a} * \log n)$.
3. If $f(n) = \Omega(n^{\log_b a+\epsilon})$, then $T(n) = \theta(f(n))$.

$\epsilon > 0$ is a constant.

Master Theorem

Each of the above conditions can be interpreted as:

1. If the cost of solving the sub-problems at each level increases by a certain factor, the value of `f(n)` will become polynomially smaller than $n^{\log_b a}$. Thus, the time complexity is oppressed by the cost of the last level ie. $n^{\log_b a}$
2. If the cost of solving the sub-problem at each level is nearly equal, then the value of `f(n)` will be $n^{\log_b a}$. Thus, the time complexity will be `f(n)` times the total number of levels ie. $n^{\log_b a} * \log n$
3. If the cost of solving the subproblems at each level decreases by a certain factor, the value of `f(n)` will become polynomially larger than $n^{\log_b a}$. Thus, the time complexity is oppressed by the cost of `f(n)`.

Master Theorem

Solved Example of Master Theorem

$T(n) = 3T(n/2) + n^2$

Here,

$a = 3$

$n/b = n/2$

$f(n) = n^2$

$\log_b a = \log_2 3 \approx 1.58 < 2$

ie. $f(n) < n^{\log_b a + \epsilon}$, where, ϵ is a constant.

Case 3 implies here.

Thus, $T(n) = f(n) = \theta(n^2)$

Master Theorem

Master Theorem Limitations

The master theorem cannot be used if:

- `T(n)` is not monotone. eg. `T(n) = sin n`
- `f(n)` is not a polynomial. eg. `f(n) = 2^n`
- `a` is not a constant. eg. `a = 2n`
- `a < 1`

Example Consider the recurrence relation: $T(n) = 3T\left(\frac{n}{2}\right) + n^2$.

Here, $a = 3$ (number of subproblems), $b = 2$ (factor by which the input size is reduced), and $f(n) = n^2$ (cost excluding recursive calls). Now, we compare $f(n) = n^2$ with $n^{\log_b a} = n^{\log_2 3}$. Since $f(n) = n^2$ is $O(n^{\log_2 3 - \epsilon})$ for any $\epsilon > 0$, we are in **Case 1**. Therefore, the time complexity is $T(n) = \Theta(n^{\log_2 3})$.

Example Consider the recurrence relation: $T(n) = 2T\left(\frac{n}{2}\right) + n \log n$.

Here, $a = 2, b = 2$, and $f(n) = n \log n$. Compare $f(n) = n \log n$ with $n^{\log_b a} = n^{\log_2 2} = n$. Since $f(n) = n \log n$ is $\Theta(n^{\log_2 2})$, we are in **Case 2**. Therefore, the time complexity is $T(n) = \Theta(n \log n \log n)$.

Example Consider the recurrence relation: $T(n) = 4T\left(\frac{n}{2}\right) + n^3$.

Here, $a = 4, b = 2$, and $f(n) = n^3$. Compare $f(n) = n^3$ with $n^{\log_b a} = n^{\log_2 4} = n^2$. Since $f(n) = n^3$ is $\Omega(n^{\log_2 4 + \epsilon})$ for any $\epsilon > 0$, and $af\left(\frac{n}{b}\right) = 4\left(\frac{n}{2}\right)^3 = \frac{1}{2}n^3 \leq kf(n)$ for $k = \frac{1}{2}$ and sufficiently large n, we are in **Case 3**. Thus, the time complexity is $T(n) = \Theta(n^3)$.

Example Consider the recurrence relation: $T(n) = 2T\left(\frac{n}{2}\right) + n \log n$.

Here, $a = 2, b = 2$, and $f(n) = n \log n$. Compare $f(n) = n \log n$ with $n^{\log_b a} = n^{\log_2 2} = n$. Since $f(n) = n \log n$ is $\Theta(n^{\log_2 2})$, we are in **Case 2**. Therefore, the time complexity is $T(n) = \Theta(n \log n \log n)$.

Problems :-

1. $T(n) = 9T(n/3) + n$
2. $T(n) = T(2n/3) + 1$
3. $T(n) = 3T(n/4) + n \log n$
4. $T(n) = 2T(n/2) + n \log n$
5. $T(n) = 2T(n/2) + \Theta(n)$
6. $T(n) = 8T(n/2) + \Theta(n^2)$ # Matrix multiplication
7. $T(n) = 7T(n/2) + \Theta(n^2)$
8. $T(n) = 2T(n/4) + 1$
9. $T(n) = 2T(n/4) + \sqrt{n}$
10. $T(n) = 2T(n/4) + n$
11. $T(n) = 2T(n/4) + n^2$

Let us take $n = 2^m$. Then we have the recurrence

$$T(2^m) = 2T(2^{m-1}) + 2^m \log_2(2^m)$$

$$= 2T(2^{m-1}) + m2^m$$

Let $T(2^m) = f(m)$

$$f(m) = 2f(m-1) + m2^m$$

$$= 2(2f(m-2) + (m-1)2^{m-1}) + m2^m$$

$$= 4f(m-2) + (m-1)2^m + m2^m$$

$$= 4(2f(m-3) + (m-2)2^{m-2}) + (m-1)2^m + m2^m$$

$$= 8f(m-3) + (m-2)2^m + (m-1)2^m + m2^m$$

$$\vdots$$

$$f(m) = 2^m f(0) + 2^m (1+2+3+\cdots+m)$$

$$= 2^m f(0) + \frac{m(m+1)}{2} 2^m$$

$$= 2^m f(0) + m(m+1)2^{m-1}$$

$$T(n) = nT(1) + n\left(\frac{\log_2(n)(1+\log_2(n))}{2}\right)$$

$$= \Theta(n \log^2 n).$$

Example (Solving the Recurrence Relation of Merge Sort):

Applying the Master Theorem, we have $a = 2$, $b = 2$, and $f(n) = n$. The recurrence relation falls into the second case of the Master Theorem.

$$T(n) = \Theta(n \log n)$$

This means that the time complexity of Merge Sort is $\Theta(n \log n)$ in the worst, average, and best cases.

Example: Let us analyze the time complexity with a numerical example using the array [38, 27, 43, 3, 9, 82, 10].

1. **Divide (logarithmic):** Dividing the array into sublists until each sublist contains one element ($\log_2 7 \approx 2.81 \Rightarrow 3$ levels of recursion).
2. **Conquer (constant):** Sorting each sublist containing one element.
3. **Combine (linear):** Merging the sorted sublists ($O(n)$ for each level of recursion).

The overall time complexity is $O(n \log n)$ for this example.

Merge Sort's efficiency lies in its consistent and predictable time complexity of $O(n \log n)$. This makes it suitable for sorting large datasets, and its divide-and-conquer strategy ensures a balanced and efficient approach to sorting. While Merge Sort incurs a higher space complexity due to the need for additional memory during merging, its time complexity makes it a popular choice for various applications.

TREES

> It is a non-linear data structure.
> It is mainly used to represent data containing a hierarchical relationship between element e.g. table of contents.
> Imp: No loop should be formed

* Binary Trees:

A binary tree T is define as a finite set of elements, called nodes, such that
(a) T is empty (called the null tree or empty tree) or
(b) T contains a distinguished node R, called the root of T, and the remaining node of T forms an ordered pair of disjoint binary tree T_1 and T_2.